Rstudio Tutorial and Lab

Note: The bulk of this tutorial is adapted from https://whitlockschluter3e.zoology.ubc.ca/Tutorials%20using%20R/index.html. Please follow this link for an even *more* in-depth look at R, more practice problems, and helpful diagrams of R syntax/commands.

Getting Started

- Opening RStudio
  - We can troubleshoot any issues with downloads here. Every computer is different.
- File types:
  - R Script - most basic. This is where the whole thing can run as code. I like to use this one, but it does have disadvantages (you have to remember to put comments in with hashtags)
  - R Notebook - This is a little more like a traditional file, but your code has to run in "chunks". I don't prefer it, but it looks nicer when you export it.
  - R Project - This isn't a file so much as it is an organization structure. When you make this, all of your scripts, etc. fall under this and are easy to reopen and keep working on. This probably won't be necessary for this class, but if you were starting a very large long term project, it can be helpful.
- Navigating RStudio: You have 4 windows.
  - Top Left: Your script. This is where you type! You can edit this all you want, AND save it for later.
  - Bottom Left: Your console. This is where all of the action happens. Here, R is actually running your code and acting on it. You *can* type in here, but you can't save anything for later!
  - Top Right: Your environment. This is where all of the dataframes and lists go. If you import data you collect (from a CSV), it will go here, and you can use commands in R to edit it, and can perform analyses on it.
  - Bottom Right: Plot viewer (and others). Mainly, this is used for viewing plots when you run code from a script. If you run code to create a visual, it will pop up here. You can also use this to view all the files visible to R in your working directory, and view all of the R packages that you have installed and loaded.
- First, let's try playing around.
  - **If we want to save our work for later, where should we type?**
  - Type 2+2 and hit enter. It should give you the answer 4.

    - The other basic calculations are the forward slash for division, the minus sign for subtraction, and the asterisk for multiplication.
    - Parentheses can be used to order your calculations (PEMDAS). Try 10+(8/2). You should get 14.
    - Type log(42) and hit enter. It should give you the answer 3.74ish.
        - Note: log() is a function. This tells R to perform the function "log" on the argument within the parentheses (in this case, 42).
        - Note: in R, log() gives you the natural log of the number.
    - Type sqrt(9). You should get an answer of 3.
    - If you want an exponent, you should use the ^ symbol. Type 4^3. You should get 64.
    - You can combine all of these things to do basically any mathematical formula. **I'll put a challenge problem on the board, and you'll type the formula into R to get the right answer.**
- Comments: try putting some comments into your code with hashtags! These are lines that will not run as code, so you can just type whatever you want. I use them as reminders to myself of what each chunk of code means, so I don't forget later.
- Functions:
    - Has a name and 1+ arguments.
    - log(4) is a function that calculates the log (base e) of 4. Name is log, argument is 4.
    - This function has optional input. The base of the log - the default is e, so when we don't specify, R assumes that we want log base e. To get a different base, we can specify in an argument:
        - Compare log(4) to log(4, base = 10).
- Variables
    - You can store information in variables! To assign a value to a variable, you use an arrow like this "<-". "X <- 5" will assign the value 5 to X. It assigns the value on the right to the variable on the left. Assign X a value of 5, and then type X+5. You should get 10.
    - You can also reassign values to variables. **Reassign X with a value of 3.**
    - You should see all of your variables appearing in the top right of your screen, in your environment.
- Naming things in R
    - This is extremely flexible. The general rules are "no spaces" and "descriptive but not too complex". If things are too complex, they get confusing and hard to type out, but you want them to be descriptive enough that you can differentiate them.
    - *R is case-sensitive!!!!!!* This means that a variable called "weights" and one called "Weights" are completely different. It makes sense from a "computer" standpoint, but can be very frustrating and is constantly a source of errors.
- Vectors

    - c() is a function that creates a vector, containing the list of items given in its arguments. To help you remember, you could think of the function c() meaning to “combine” some elements into a vector.
    - All you do is list all of the numbers you want in the vector, with commas between them.
    - c(1,3,5,7,9)
    - We can assign a vector to a variable as well. **Assign the numbers 78, 85, 64, 54, 102, and 98.5 to a vector with the name TemperatureF.**
    - You can then view the contents of the vector TemperatureF just by typing out its name.
    - You can also do the same calculation on all of the pieces of a vector at once, by using the name of the vector! **Let's convert our temperatures to Celsius by subtracting 32 from the values of F, and then multiplying by 5/9. Let's save this in a vector called TemperatureC.**
    - To pull out a number from a vector, add square brackets after the vector name, and inside those brackets put the "index" of the element we want. (The “index” is just a number giving the relative location in the vector of the item we want. The first item has index 1, etc.)
        - **Pull out the third number in TemperatureC.**
    - You can do operations between vectors as well. **Create a vector named "TempDiff" by subtracting the Celsius temperatures from the Fahrenheit temperatures, as an example.**
- Basic Calculations
    - You can do calculations on vectors to understand them better!
    - mean() will give you the mean value of the vector, sum() will add up all of the numbers in a vector, and length() will tell you how many elements are in a vector. **Try these on TemperatureC.**
- Packages
    - The best thing about R is packages, which other people have made and expand the functions of R by a LOT! There are tons of packages with tons of different uses, but two really common ones are ggplot2 and dplyr.
    - The pane on the lower right has an option for packages, and a button for installing them. This is extremely straightforward as long as you have the name of package in mind.
    - To actually use the package, you have to load it at the start of each new R session when you open the software.
        - You can do this with the function library(), where you just type the name of the package in as the argument, or by scrolling through the list of packages in the bottom right panel, and making sure they are selected.
        - If you use a function from a package you have installed but not loaded, it will give you an error (and often ask if you want to install or load the package).

- Setting the Working Directory
  - This is possibly the most irritating thing about R. If you want to load data into R from a CSV on your device, you have to tell R where to look for it with extreme specificity.
  - My recommendation is, for every project you embark on, to hold all of your files for it in one specific folder so you don't have to go looking for them. R can only work with one directory at a time, so if you have 10 different data files in 10 different folders, that does pose a problem. R projects help a bit with this.
  - My preferred way is to use session -> set working directory -> choose directory, and then move through my folders to find the one I want.
  - You can also use the function setwd(), with the argument being your folder name (where you keep all your files and documents of interest) *in quotations!* It must be in quotations otherwise R gets confused.
- Entering data from your files
  - To use data you've got in your files as CSVs, all we have to do is read it in with a simple function. Just make sure assign it to a named object!
  - Your working directory should already be the folder where the CSV is contained, so the function read.csv() will work. Make sure to put the name of the CSV in quotation marks! It has to be the *whole* name too, .csv and all.
  - **Read in the file "titanic.csv", and give it the name titanicData.**
- Dataframes
  - To view all of your titanic data, you can click on it in the top right panel.
  - You can also use the summary() function, which lists all of the variables and some summary stats. **Try this with the titanic data.**
  - You can also the classes of the columns of data. Here, we have character and numeric classes.
  - You can specify columns from a dataframe with $. If you just want to show a vector of the age of the passengers, type titanicData $ age.
  - If you want a new column in the dataframe, assign a new vector to a new column name.
    - **Try creating a new column in this dataframe of the log age of the passengers.** You can see if it worked by just viewing your dataframe.
  - If we want to look only at data that fit certain criteria, we can use the filter function from the dplyr package.
  - Together, let's make a new data frame w/ only the data from females. The double equals sign tells R to evaluate this statement as true or false. For the rows where the sex is female, take that data and put it in a new dataframe.

Basic Summary Statistics

- Working with NAs
  - Let's keep working with the titanic data. Go ahead and view the age column of that data.
  - You can see that quite a few of them have NA as their age. This means we don't know what their age was!
  - **Try taking the mean of titanicdata $ age. It will return NA, because we don't know some of the ages!**
  - **If we put the argument na.rm into the function, and make it equal to TRUE, it tells R to take out all of the NAs before giving us the mean. Try it!** It should return a value of about 31.2.
  - **Try getting the median age.** It should return 30.
  - The summary function will give you these values and some others, and you don't have to add that extra argument. Try it!
- Measures of variability
  - The variance function is var() and the standard deviation function is sd(). You can get the IQR with.... you guessed it, IQR(). **Try them out, and don't forget the na.rm argument!**

About plots

You can make plots just with base R, or with a package called ggplot2. Base R plots are perfectly fine, but ggplot2 makes nicer looking plots, and is often more functional. The only downside is the "language" takes a bit of learning. It works best with a dataframe where rows are observations and columns are variables.

Getting started:

- ggplot() the function is pieced together with + signs.
- the first argument in the actual ggplot() function is the data. ggplot(data=dataframe).
- the next argument in the function is the mapping argument. This tells ggplot how parts of the data are mapped onto aesthetic attributes of geometric objects.
  - How this looks: ggplot(data=dataframe, aes(x = --, y = --))
  - Think - how would I place my variables on the graph I'm making?
- The final essential argument is an extra layer. You'll add it after a plus sign to the end of your code (often using another line). Generally, this is where you'll tell ggplot what kind of graph you're making.
  - ggplot(data=dataframe, aes(-----)) + geom_type of graph()

Let's try something very easy. Let's make a histogram of the titanic passenger ages.

- our data is the titanic data.

- we just specify the X variable as age, as that is just our one continuous variable.
- we can use geom_histogram() to specify that we want a histogram.

Let's try to make a barchart of the number of male and female passengers on the titanic.

- our data is the titanic data.
- we just specify the X variable as sex - our one discrete variable.
- geom_bar() specifies that we want a histogram.

Now, let's try a boxplot of age by the sex of the passengers.

- our data is the titanic data.
- we want to separate age by the sex of the passenger. Thinking about how we want our boxplot to look, we would want two separate boxes (male and female). we should specify our x variable as sex, and our y variable as age.
- geom_box() specifies we want a boxplot.

There are a bunch of ways that we can use ggplot, more than can be listed out here. If you have a specific idea for a plot, you can do some research into all of the options that ggplot is able to work with.

Comparing means of two groups

- Two sample t-test
  - For this, we'll use the titanic data again!
  - We assume here we have data in the “long” format
    - Each row describes a different individual
    - Columns correspond to variables (age, sex, etc.)
  - For a 2-sample t-test, two variables are used, one categorical and one numerical.
  - With this data format, we can most easily give input to the t.test() function using what R calls a “formula”.
    - response variable (numerical variable) ~ explanatory variables (categorical variable defining the groups)
  - We want to test whether the individuals which survived the Titanic disaster had the same average age as those passengers who did not survive.
  - To do a 2-sample *t*-test, there are two other things to input. You need to specify which data frame contains the data, and you need to specify that you want to assume that the variances of the two groups are equal.
    - When you specify this you are telling R to do a classic 2-sample t-test rather than a Welch’s t-test.

        - To specify the data frame to use, we give a value for the argument “data”, such as “data = titanicData”. To tell R to assume that the variances are equal, we use the option “var.equal = TRUE”.
        - If we don't specify var.equal = TRUE, it will automatically assume FALSE and perform a Welch's t-test. This is the default for R.
        - A two-sided test is also the default for R!
    - **Let's test both of the t-tests out together with the titanic data**.
- Paired t-test
    - I will assume you have a dataframe with the rows being observations, and the columns being the variables. For example, you have 50 cats you observed, and collected data on their heart rate before being given catnip and after being given catnip.
    - My t-test formula would be: t.test(catdata $ HRbefore ~ catdata $ HRafter, paired = TRUE)
        - R calculates the mean of the difference by subtracting the variable you listed second from the variable you listed first.
- Levene's test of equal variance
    - You need to install and load the "car" package
    - The formula is leveneTest(), and you need three arguments:
        - data: the data frame that contains the data
        - formula: an R formula that specifies the response variable and the explanatory variables
        - center: how to calculate the center of each group (for the standard Levene’s test, use center = mean)
    - Let's try this with the titanic data, with age as the numerical/response variable and survive as the group/explanatory variable.
    - If the p-value is significant, reject the null hypothesis that the group that survived and the group that died had the same population variances in terms of their age (the population variances are not equal). And remember, the Welch's t-test doesn't assume equal variance!
- Shapiro-Wilk Normality Test
    - we can use the function shapiro.test() and enter the name of our data as the argument.

ANOVA

Let's take another look at the Titanic data. We can group passengers by the class they travelled under (categorical) and ask whether classes differed in their mean age (numerical).

First, we can try making some plots for this - just histograms. Does a histogram of age look different if we separate passengers by their class?

- we've made a histogram of the age of titanic passengers before.
- all we need to do is add another line after the geom_histogram() line with another + sign
- we will use the facet_wrap() function. this divides the plot into subplots based on 1+ discrete variables... like passenger class.
    - let's add the line facet_wrap(~passenger_class, ncol = 1) to our histogram. We already specified that we are making a histogram of the age, but this tells us to subset this into facets by class (~passenger_class). ncol = 1 just tells it to display this all in one column. take it away and you'll see it looks a bit worse.
- this visualization can also help us check some of our assumptions of ANOVA.

We'll use the dplyr package here. Make sure it's loaded!!

Organizing and Summarizing

- group_by(): organize a data frame by an appropriate grouping variable.
    - organize titanicData by passenger_class
- summarise(): apply any type of function that summarizes data (like mean(), sd(), etc.), and receive a summary by groups.
    - You can use summarise() for many summary functions at once.
- If we do this on our titanic data, we can see that the standard deviations are very similar. The data fits the equal variance assumption.
- The code will look like this:

```
#group by
titanic_by_passenger_class <- group_by(titanicData,passenger_class)

#summarise
summarise(titanic_by_passenger_class, group_mean = mean(age, na.rm=TRUE),
group_sd = sd(age, na.rm=TRUE))
```

The ANOVA model

- The first thing to do is fit an ANOVA model to the data using the function lm().
    - lm() needs a formula and a data frame
    - model formula: a response variable ~ at least one explanatory variable
    - for ANOVA: numerical variable ~ categorical variable
    - For us to compare differences in mean age among passenger classes on the Titanic: age ~ passenger class
    - Here, it is necessary to save the intermediate results by assigning them to a new object, which anova() can then use to make the ANOVA table.
        - Let's assign the results of lm() to a new object named “titanicANOVA”

        - We can do this all in one step, by putting titanicANOVA to the left of an arrow and lm() to the right of that arrow.
- Second, we use the results of first step to do the ANOVA calculations and place them in an ANOVA table using the function anova().
    - The function anova() takes the results of lm() as input and returns an ANOVA table as output.
        - We named our results titanicANOVA and can just use that as the argument for the anova() function!
    - This table shows the results of a test of the null hypothesis that the mean ages are the same among the three groups.
    - The *P*-value is very small, and so we reject the null hypothesis of no differences in mean age among the passenger classes.

Tukey-Kramer test

- Apply the function aov() to titanicANOVA
- Apply the function TukeyHSD to the result
    - Can do this all in one step! TukeyHSD(aov(titanicANOVA))
- Gives us a table estimating the difference between the means of groups and calculates a corresponding 95% confidence interval
- Gives the P-value from a test of the null hypothesis of no difference between the means (p adj)
    - Where P is less than 0.05, the population mean ages of passenger classes are significantly different from each other.

Kruskal-Wallis test

- Non-parametric analog of a one-way ANOVA.
- Use the function kruskal.test()
    - The input for this function is the same as we used for lm() above.
- (This test also rejects the null of equal ages for all passenger class groups)

Regression

"guppydata.csv" investigates the relationship between how ornamented a father guppy is and how attractive to females are his sons.

Correlation

- cor() calculates the correlation between the two variables given as input.
    - The input is simply the names of the vectors of the two variables

    - (Can be vectors, or columns in a dataframe!)
- cor.test() tests a hypothesis about the correlation coefficient / calculates its confidence interval
    - Input: names of vectors containing the variables of interest, just like above
    - Spearman's rank correlation can be calculated if we add the option method = "spearman" as an argument in the function

Regression

- lm() for regression similar to ANOVA!
- response variable ~ explanatory_variable
    - Want to predict son's attractiveness from father's ornamentation: sonAttractiveness ~ fatherOrnamentation
- The other argument is our dataframe name.
- Assign the results of the lm() to an object name so you can use it later (with functions like summary())
- If you call the object name, you receive the coefficients of the intercept and predictor variable. This gives us: sonAttractiveness = 0.005084 + (0.982285 x fatherOrnamentation)
- Calling summary() on the object name gives you all of the relevant information about the model, including the p-value.
- Plotting:
    - ggplot
    - Add the layer "+ geom_smooth(method=lm)", which adds
        - best-fitting regression line
        - 95% confidence interval for the line (confidence bands)

Plotting Simple Regression

```
ggplot(data = guppyData, aes(x=fatherOrnamentation, y=sonAttractiveness)) +
# we are using guppyData, our predictor (father) and our response (son)
  geom_point() + # tells ggplot we want a scatterplot of these 2 variables
  theme_minimal() + # purely aesthetic
  xlab("Father's ornamentation") + # labels for your plot
  ylab("Son's attractiveness") +
  geom_smooth(method = lm) # adds trendline and confidence interval band
```

Residuals

- residuals() function: give it the results from the lm() function (like the object name from above)
- Residual plot: plot the residuals of each data point as a function of the explanatory variable

- The plot() function is all we need here (residuals ~ explanatory variable)
- abline(h=0) adds a horizontal line at 0

Multiple Regression
Fish size might depend on water temperature, depth, and algae cover, or the type of site at which the fish are found.
"FishSizeData.csv" can help us take a look at this.

Data Inspection

- These functions will provide you with a scatterplot and correlation matrix. They only work with numeric data though!
  - pairs(fishdata[, c("size", "temp", "depth", "algae")])
  - cor(fishdata[, c("size", "temp", "depth", "algae")])
  - can also try a histogram of fish size with hist()

Modelling:

- Just like single regression, but add plus signs between the different explanatory variables.
  - The formula reads as size is modeled as a function of temp plus depth plus algae
  - Make sure to assign your model to an object name!
- The summary() outputs estimates of the coefficients, P-values for all of them, the r2 and adjusted r2, and the f-statistic and associated P-value.

Assumptions

- To check residuals vs. fitted/predicted and your qq plot, calling plot() on your model object name and adding the argument "which = 1" for the former and "which = 2" for the latter will show you those plots so you can take a look at them.
- Collinearity
  - The function vif() with the argument of the object name of your model will give you the VIFs of your predictors.

A second model:

- R can handle placing categorical variables into a model.
- If you add "site" into your model with another + sign, the results from summary() will show the coefficient "sitereserve"
  - This is the difference in mean size between sites that are "reserve" and sites that are the baseline level "open", holding the other variables constant. (R uses the alphabetically first level/option as the baseline - so between the two options for sites, "reserve" and "open", open comes first.)

AIC

- AIC stands for Akaike Information Criterion.
- It is a popular way to compare models relative to each other.
- The score balances the fit of the model to the data against the complexity of the model.
- The model with the lowest AIC score is generally selected.
- Calling AIC() with the arguments being the names of your model objects (with commas between them) gives you a simple list of their AIC scores to look at.

Assignment

T-Testing

Part 1
Scientists are interested in the efficacy of marine reserves on a rare species of abalone. To test this, the researchers paired each reserve with a nearby non-reserve site of similar ecological features and size. They then counted the rare abalone within the reserve and the paired non-reserve site.

For this, use the abalone_reserve.csv data.

Data Exploration: What are the mean counts inside and outside the reserve? What are the standard deviations? (1 point)

Make a very simple boxplot for visual comparison: Do the data appear normally distributed? Do the distributions look similar? (1 point)
Hint: for this, use the base R function boxplot() for ease. Use the second and third column of the dataframe (dataframename[,2:3]) as the argument in the function. Base R does have plotting functionalities, and they can work better in some cases!

Run a t-test and report the relevant results. What can you infer about how effective the reserves are? (2 points)

Part 2
We are going to take a look at Yellowfish length data from two separate marine sites: La Jolla and Point Loma. We will evaluate whether the two populations are of different size distributions.

For this, use the yellowfish.csv data.

Before you start, use the as.factor() function to manipulate the data:

```
dataframename$Population <- as.factor(dataframename$Population)
```

Data Exploration: What are the mean lengths of Yellowfish in each location? What are the variances? Do they meet the assumption of equal variances with a formal test? (2 points)
Hint: One of the easiest ways to get separate means and variances in this case may be with the filter function.

Create a simple boxplot to visually compare the treatments. (1 point)
Hint: with your original dataframe, you can use ggplot, or you can try base R.

Make a histogram of yellowfish size subsetted by the population they are from. Do the data appear normally distributed? Can you test this formally? (2 points)

Run a t-test and report the relevant results. What can you infer about the Yellowfish populations? (2 points)

ANOVA

To monitor water quality along the San Diego coastline, scientists and managers sample water from several popular beaches to ensure safety for surfers and beachgoers. For each sample they measure the bacterial cell count in a milliliter and then calculate a normalized score, where a higher value indicates a higher bacterial load.

For this, use the bacteriadata.csv file.

When you read in your data, use the as.factor() function and transform the "Beach" column into a factor.

Create a simple boxplot of the bacteria levels by treatment group (beach). Do any of the beaches appear different from the others? (1 point)

Create the ANOVA model and run it. Report the pertinent results. (1 point)

Now let's run a post-hoc test to see which treatment(s) are driving any patterns we observe (a Tukey-Kramer test). Report and discuss your results. (1 point)

Regression

We might have figured out a good way to age lions using pictures of their noses, but what about sharks? Animals like great white sharks are difficult to monitor, but scientists are trying to come up with the best way to accurately age them. They manage to get 100 samples from sharks for whom they miraculously know the true age. They also collect data on:

- number of spots on their gills - perhaps they gain more over the course of their life.
- number of vertebral bands - could be used like rings of a tree trunk.
- total length (cm) - sharks grow their whole life, perhaps their length can be used as a proxy.
- telomere length - human telomeres get shorter with age, it could be the same for sharks.

- number of scars - they collect scars during the course of their life through rough interactions.

For this, use the SharkAge.csv file.

Take a closer look at your data, using a correlation table and other things if you so choose. Which variables seem to be most strongly correlated? (1 point)

Pick 2 of these predictor variables for age and create a model with them. Check assumptions (residuals vs. fitted, qqplot, VIF), and report the relevant results. (1 point)

Add another 2 predictor variables to a second model. Check assumptions, and report the relevant results, comparing them to the first model you made. (2 points)

Compare the AIC values of your models. According to this, which one would you use? (1 point)

Choose **A SINGLE** explanatory variable to make a scatterplot with comparing it to the age of a shark. Make sure to add in a trendline. This is just for practice editing the code, so it doesn't matter which one of the explanatory variables it is - pick your favorite! (1 point)

Deliverables

Some sort of document answering the questions from the assignment portion, and R file(s). You can create a word/google doc with the answers to your questions and the plots as images, or whatever you think will work best for you.

This assignment is worth 25 total points. 20 points for the assignment portion at the end, and 5 additional points for sending your R file(s) - *including* the lab walkthrough with the code you practiced with.

Sample code for the R tutorial/walkthrough from lab will be available 24 hours after the lab.

Sample code for the assignment will be available after the lab due date.

This assignment is due on Sunday, Oct. 25 at 11:59 PM.